

Introduction to Compilers

Problem Set: Tokens & Regular Expressions (Lecture 2)

Instructions. You have the full class period; the set itself is designed to take about 90 minutes, so budget some slack. Work individually. Show your work — partial credit is available on every problem. Submit your answer sheet at the end of class. Approximate time budgets are given for pacing only; they are not a hard limit.

Problem 1 — Classify the tokens (~12 min)

Classify every token in the line below by class (keyword, identifier, integer, float, operator, or punctuation):

```
while (n >= 1.0) total = total + n;
```

Problem 2 — Track the position (~15 min)

Consider the following 3-line source file (1-based line and column numbering; each line ends with a newline):

```
sum = 0
count = 10
sum = sum + count
```

- (a) What line and column would the lexer report for the token `count` on line 2?
 - (b) What line and column would the lexer report for the `+` on line 3?
 - (c) In your own words, explain why a lexer tracks position character-by-character while scanning, rather than computing it afterward.
-

Problem 3 — Find and fix the bug (~20 min)

A student writes the following hand-written code to lex a string literal (input starts right after the opening `"`):

```
StringBuilder sb = new StringBuilder();
while (true) {
    int c = input.read();
    sb.append((char) c);
    if (c == '"') break;
}
return new Token(TokenType.STRING, sb.toString());
```

- (a) Trace this code on the input `hi"` (i.e. the two characters `h`, `i`, followed by the closing quote). What string ends up inside the returned token?
 - (b) Identify the bug, and explain why it causes that incorrect result.
 - (c) Rewrite the code so it produces the correct token content.
-

Problem 4 — Write the regex

(~20 min)

Using JFlex-style syntax and the abbreviations `digit = [0-9]` and `hexdigit = [0-9a-fA-F]`:

- (a) Write an abbreviation `hexlit` for hexadecimal integer literals such as `0x1A3F`, `0xff`, and `0x0` (the literal characters `0x`, followed by one or more hex digits).
 - (b) Give two strings that `hexlit` matches, and two strings that look similar but do *not* match it.
 - (c) Is `0X1A` (capital `X`) matched by your `hexlit`? If not, revise your abbreviation so that it accepts both `0x` and `0X` prefixes.
-

Problem 5 — Put it together

(~25 min)

You are writing the lexer for a tiny language whose only statement form is a simple assignment: an identifier, then `=`, then an integer literal, then `;` — for example `count = 42;`.

- (a) Write a JFlex specification fragment (abbreviations plus a `<YYINITIAL>` block with patterns and actions) that tokenizes this language. Include rules for whitespace, identifiers, integers, `=`, and `;`.
- (b) Trace your spec on the input `count = 42;`, listing the sequence of tokens produced.
- (c) Suppose your spec also had a keyword `end`. Would your lexer ever misread an identifier like `ending` as the keyword `end` followed by the identifier `ing`? Justify your answer using the tie-breaking rule(s) a lexer generator applies when more than one pattern matches.